From Data Mixtures to Model Architectures to Agentic Intelligence: Uncovering the Anatomy of the Foundation Model Development Lifecycle

Piyush Sawarkar

Founding Member & Tech Lead Text Foundation Models @ BharatGen, India's Sovereign AI National Mission 🇮🇳
IIT Bombay Alumnus

IIT Bombay • October 9, 2026

Talk Overview

- The Model Development Life Cycle
- Case Study on BharatGen's Flagship Model: Param
  - Training Dataset and Curation Process
  - Tokeniser -> Algorithms -> Fertility Comparison
  - Architecture Details
- LLM Post-Training and Off-Policy, On-Policy Interplay
- Provenance Mapping Key to Unlocking and Sustaining Performance Climb!
- Graduating LLMs to Intelligent Agentic AI Systems: Harness/Agents

Talk Overview

The Model Development Life Cycle

Case Study on BharatGen's Flagship Model: Param

- **Training Dataset and Curation Process**
- **Tokeniser -> Algorithms -> Fertility Comparison**
- **Architecture Details**

LLM Post-Training and Off-Policy, On-Policy Interplay

Provenance Mapping Key to Unlocking and Sustaining Performance Climb!

Graduating LLMs to Intelligent Agentic AI Systems: Harness/Agents

The Model Development Life Cycle

1. Data	2. Tokenizer	3. Architecture	4. Pretraining	5. Post-training	6. Evaluation
collect + curate	language encoding	model design	largest stage	SFT + RL	benchmarks + linguistic

Talk Overview

The Model Development Life Cycle

Case Study on BharatGen's Flagship Model: Param

- Training Dataset and Curation Process
- Tokeniser -> Algorithms -> Fertility Comparison
- Architecture Details

LLM Post-Training and Off-Policy, On-Policy Interplay

Provenance Mapping Key to Unlocking and Sustaining Performance Climb!

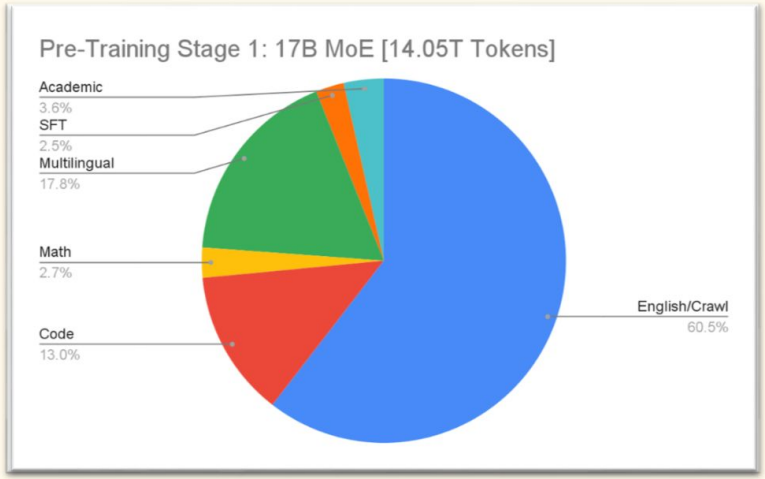
Graduating LLMs to Intelligent Agentic AI Systems: Harness/Agents

Training Dataset details for the Param 17B Foundation Model

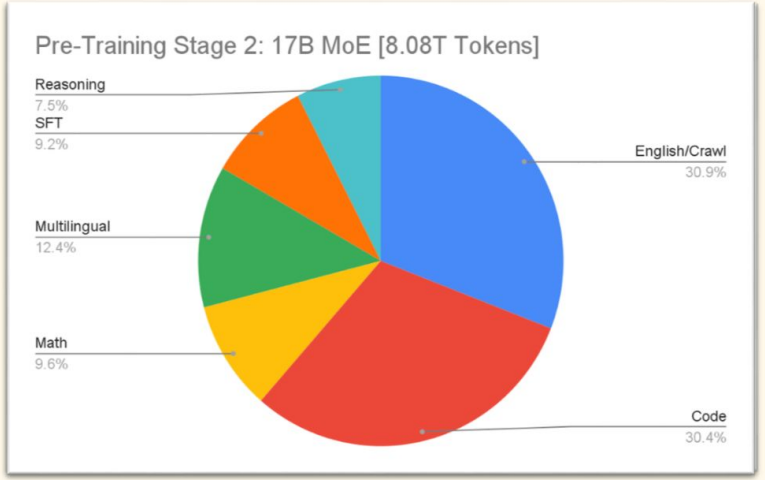
PARAM 2 17B MoE - DATA DETAILS

Dataset Preparation Details: Multi-Stage Pretraining on 22.13 Trillion Tokens

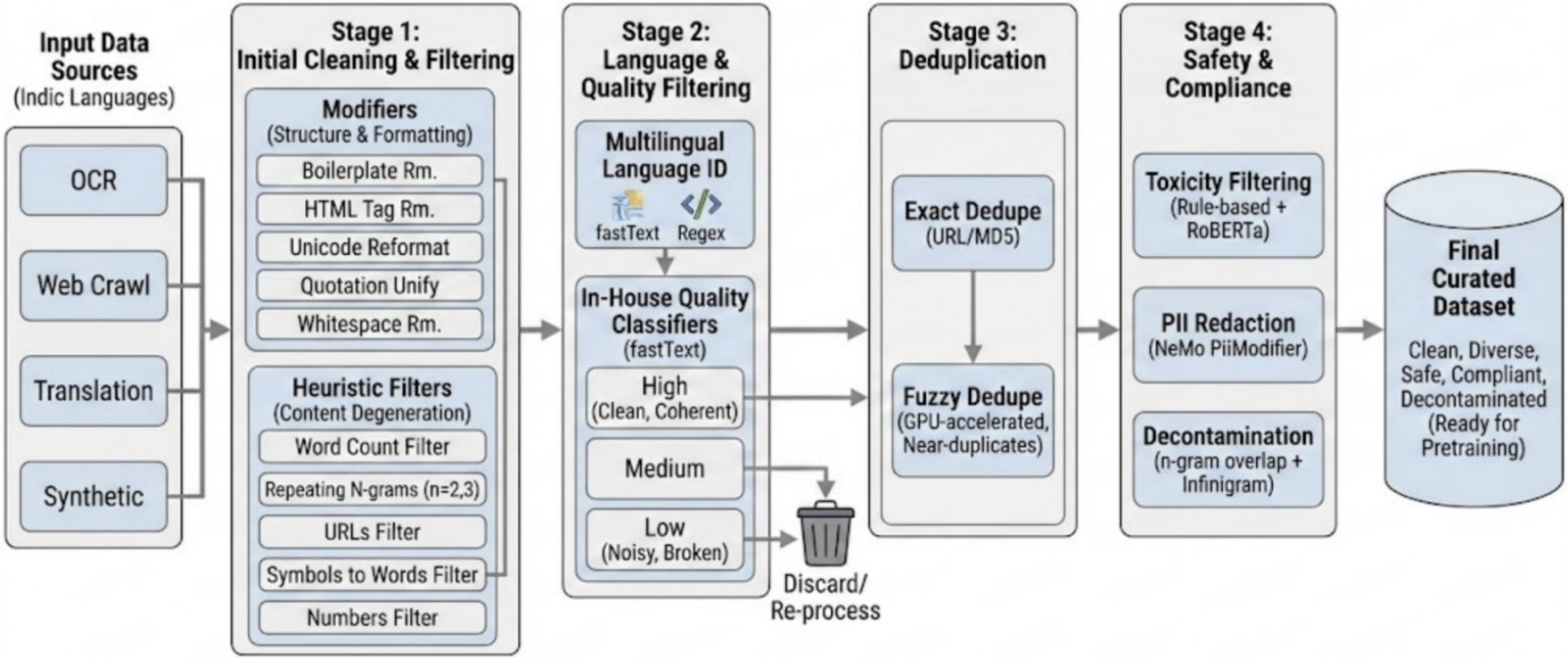
Pre-Training Stage 1 [14.05T]



Pre-Training Stage 2 [8.08T]



Data Curation Process



Data Mixture Algorithms – Empirical vs Optimisation

1 Train **small-scale** proxy models

Hacker News	Github	Philpapers	Target ↓
9.5%	35.9%	54.6%	5.46
87.7%	12.0%	0.3%	5.57
24.4%	1.4%	74.2%	6.07
...			...

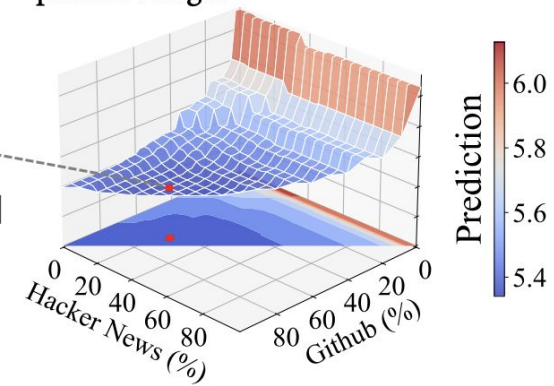
step

2 Fit a **regression** model using data mixture as source

Regression Model

Linear Model *Tree Model*

3 **Simulate** new data mixtures and predict Target



4 Train a **large-scale** model on the best mixture

Simulated Best Data Mixture

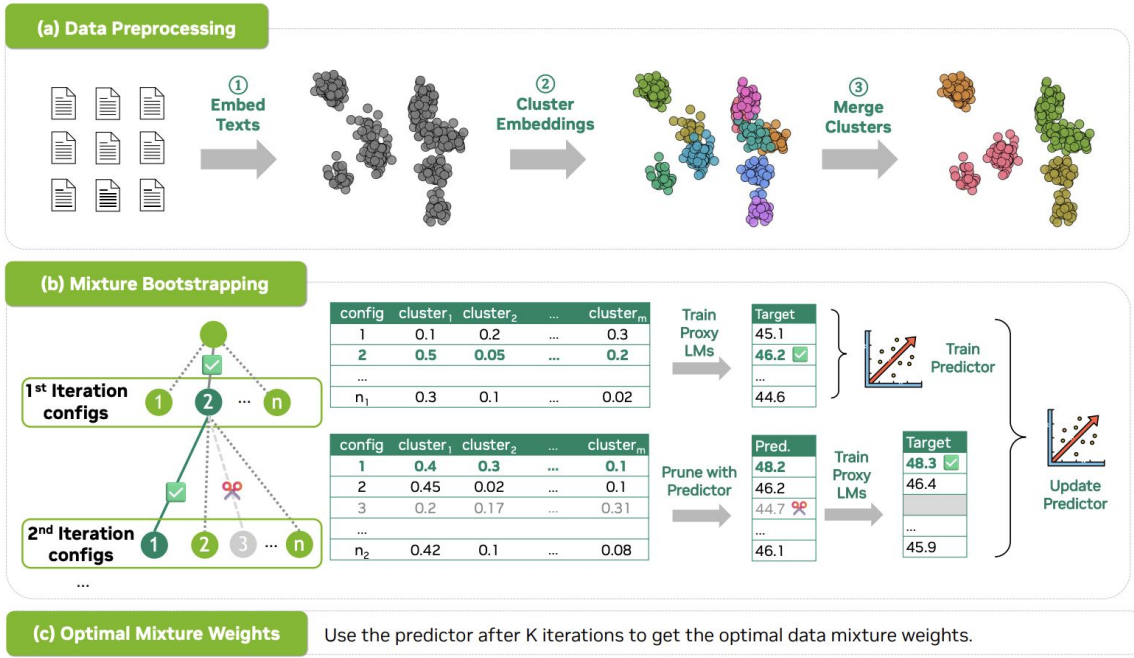
Hacker News	Github	Philpapers
22.8%	67.0%	10.2%

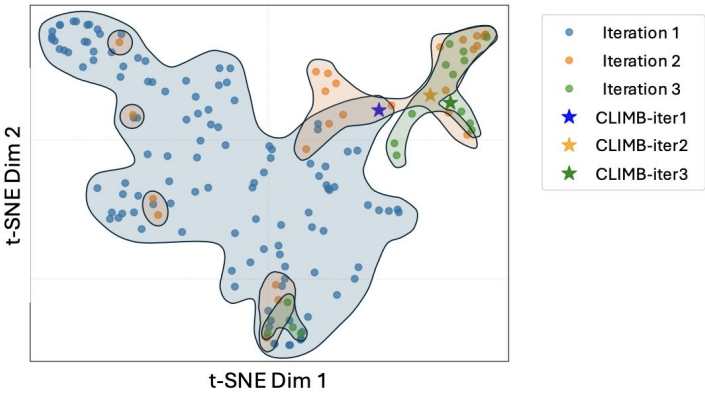
Predicted Best Target

Prediction (Lowest)
5.34

REGMIX: DATA MIXTURE AS REGRESSION FOR LANGUAGE MODEL PRE-TRAINING: https://arxiv.org/pdf/2407.01492

Data Mixture Algorithms – Empirical vs Optimisation





Nemotron-CLIMB: CLustering-based Iterative Data Mixture Bootstrapping for Language Model Pre-training: https://arxiv.org/pdf/2504.13161

Talk Overview

The Model Development Life Cycle

Case Study on BharatGen's Flagship Model: Param

- Training Dataset and Curation Process
- Tokeniser -> Algorithms -> Fertility Comparison
- Architecture Details

LLM Post-Training and Off-Policy, On-Policy Interplay

Provenance Mapping Key to Unlocking and Sustaining Performance Climb!

Graduating LLMs to Intelligent Agentic AI Systems: Harness/Agents

The Tokenizer

Many words map to one token, but some don't: indivisible.
Unicode characters like emojis may be split into many tokens containing the underlying bytes: 🤚
Sequences of characters commonly found next to each other may be grouped together: 1234567890



After Tokenization

```
Many words map to one token, but some don't: indivisible.

Unicode characters like emojis may be split into many tokens containing
 the underlying bytes: ����

Sequences of characters commonly found next to each other may be grouped
 together: 1234567890
```

The Tokenizer

A tokenizer converts raw text into numbers the model can process. It breaks words into chunks called tokens. The model never sees the original text, it only sees sequences of numbers representing tokens.

What is a token?

A token is not necessarily a word. It is the basic unit the model is trained on. OpenAI's tokenizer, for example, splits "Bombay" into two tokens: "Bomb" + 'ay". BharatGen's tokenizer treats "Bombay" as one token, because it was trained on Indian data and "Bombay" is a well-known place.

Word	OpenAI tokenizer	BharatGen tokenizer
Bombay	2 tokens: "Bomb" + 'ay" → charged 2x	1 token: "Bombay" → charged 1x
Hindi word (e.g., नमस्ते)	May split into 6–7 tokens (byte-level fallback)	1–2 tokens (trained on Indian scripts)

The Tokenizer

Fertility: the key metric

Fertility is the ratio of tokens to words. Lower is better: it means the tokenizer is more efficient at representing the language.

Fertility (English)	Fertility (Indian langs, global models)	Fertility (Indian langs, BharatGen)
~1.4	**6–7x**	**1–3x**
tokens per word (industry standard)	tokens per Indian-language word	tokens per Indian-language word

Why fertility matters commercially: API pricing is charged per token. If a global model charges $10 per 1M output tokens and uses 6–7 tokens for every Indian-language word, an Indian enterprise using it in Hindi is paying 4–5x more than they would with BharatGen's model for the same output. This is a direct cost argument when talking to clients.

Algorithm – Updating the Language Proportion Dynamically and Optimise for Fertility

For each language $l \in L$ at iteration N, a *scaled token-to-word ratio* deficit is computed to quantify tokenization inefficiency relative to a reference fertility f_{ref}, normalized by the fertility range $f^N_{\text{range}} = \max_{k \in L} f^N_k - f_{\text{ref}}$, the gap between the worst-served language and the reference.

$$\delta^N_l = \frac{f^N_l - f_{\text{ref}}}{f^N_{\text{range}}} \tag{1}$$

We fix $f_{\text{ref}} = 1.00$, a constant, for all languages and all iterations, so that δ^N_l is 0 for a language at the reference and 1 for the worst-served language in the current iteration. The apparently natural alternative, $f_{\text{ref}} = f^N_{\text{best}} = \min_{k \in L} f^N_k$, is degenerate; §5 analyses why and reports the experiment that established it.

A small constant is added to each δ^N_l to ensure all languages retain non-zero weight, preventing exclusion.

$$w^N_l = \delta^N_l + \varepsilon \tag{2}$$

The resulting deficit weights are normalized across all languages to ensure that the resulting values form a valid probability distribution. These proportions give the composition for the next iteration; languages with a higher token-to-word ratio receive larger proportions.

$$t^N_l = \frac{w^N_l}{\sum_{k \in L} w^N_k} \tag{3}$$

To avoid abrupt shifts in the sampling distribution from one iteration to the next, the target proportions are combined with the previous mixture using an exponential moving average. This results in an updated mixture computed as a weighted combination of the past and current targets, controlled by the smoothing factor μ, which determines the aggressiveness of weight redistribution. Smaller μ leads to slower changes and preserving stability, whereas a larger μ allows faster adaptation. This mechanism ensures that mixture adjustments are gradual and stable, reducing the risk of overcorrection.

$$m^N_l = (1 - \mu) \cdot m^{N-1}_l + \mu \cdot t^N_l \tag{4}$$

The updated mixture is scaled by the sampling budget T to give the number of characters allocated to each language, rounded to the nearest integer and renormalized for rounding deviations.

$$C^N_l = \text{round}(m^N_l \cdot T) \tag{5}$$

Together, these steps form a feedback loop that raises the representation of underperforming languages without destabilizing well-performing ones, and can be written as a single equation:

$$m^N_l = (1-\mu) \cdot m^{N-1}_l + \mu \cdot \left(\frac{\frac{f^N_l - f_{\text{ref}}}{f^N_{\text{range}}} + \varepsilon}{\sum_{k \in L} \left(\frac{f^N_k - f_{\text{ref}}}{f^N_{\text{range}}} + \varepsilon \right)} \right) \tag{6}$$

This formula is applied iteratively for each N, and m^N_l is re-normalized if the sum deviates from 1. If $f^N_{\text{range}} = 0$ – that is, if no language exceeds the reference fertility – the optimizer reuses the previous mixture proportions, rescaled to match the sampling budget. Since $f_{\text{ref}} = 1.00$ is unattainable, f_{range} stays well away from zero in practice (the worst-served language exceeds 2.5 in every mixture we report), so this case is a formal guard rather than an operating regime:

$$m^N_l = \frac{m^{N-1}_l}{\sum_{k \in L} m^{N-1}_k} \cdot T \tag{7}$$

Algorithm 1 AdaptMix mixture update (one iteration)

Require: current mixture m^{N-1} (characters per language), fertility scores f^N, smoothing factor μ, constant ε, reference fertility $f_{\text{ref}} = 1.00$
Ensure: updated mixture m^N

$T \leftarrow \sum_{k \in L} m^{N-1}_k$
$f_{\text{range}} \leftarrow \max_{k \in L} f^N_k - f_{\text{ref}}$
if $f_{\text{range}} = 0$ **then**
    return m^{N-1} rescaled to T
end if
for each language $l \in L$ **do**
    $\delta_l \leftarrow (f^N_l - f_{\text{ref}}) / f_{\text{range}}$
    $w_l \leftarrow \delta_l + \varepsilon$
end for
for each language $l \in L$ **do**
    $t_l \leftarrow w_l / \sum_{k \in L} w_k$
    $p_l \leftarrow m^{N-1}_l / T$
    $m^N_l \leftarrow \text{round}\big(((1 - \mu)\, p_l + \mu\, t_l) \cdot T\big)$
end for
return m^N normalized to T

Algorithm – Morphologically Grounded and Constrained Tokenisation Efforts

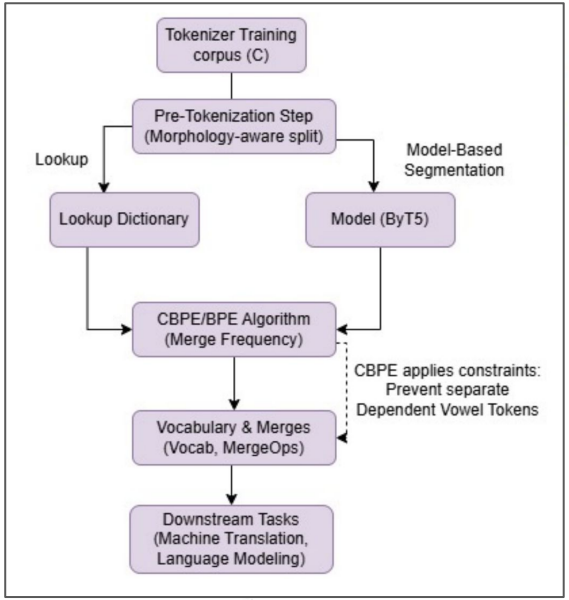
❏ Pre-tokenization + BPE: “Morphologically Grounded Tokenization for Indian Languages” - Preprint Link

Algorithm 1 Morphological grounded Tokenization

Require: [Input] Training Corpus $\mathcal{C}$; No. of Merges $\mathcal{K}$; Pre-tokenization Type $\mathcal{T}$ ($\mathcal{T}$ = Model or Lookup); Lookup $\mathcal{L}$ (consist of Word $\mathcal{W}$ and Segments $\mathcal{S}$ pairs)
Ensure: [Output] Vocabulary $\mathcal{V}$, Merges $\mathcal{M}$

$\mathcal{C}' \leftarrow$ PreTokenize($\mathcal{C}$, $\mathcal{T}$)
$\mathcal{V}, \mathcal{M} \leftarrow$ BPE($\mathcal{C}'$, $\mathcal{K}$) ▷ Learn Merges using BPE

procedure PreTokenize($\mathcal{C}$, $\mathcal{T}$)
  if $\mathcal{T}$ equals Model **then**
    $\mathcal{U} \leftarrow$ ExtractUniqueWords($\mathcal{C}$)
    $\mathcal{D} \leftarrow$ WordSegmentationModel($\mathcal{U}$)
    $\mathcal{D}' \leftarrow$ Filter($\mathcal{D}$)
    $\mathcal{C}' \leftarrow$ Replace($\mathcal{W}, \mathcal{S}, \mathcal{C}$)
  else
    $\mathcal{D} \leftarrow$ ReadLookup($\mathcal{L}$)
    $\mathcal{C}' \leftarrow$ Replace($\mathcal{W}, \mathcal{S}, \mathcal{C}$)
  end if
  return $\mathcal{C}'$
end procedure



Word	BPE Segments	Morphologically Grounded Segments
खुलता	खु लता	खुल ता
उपजता	उप जता	उपज ता
कांडला	का ंड **ला**	कांड **ला**
गोलार्ध	**गोल** ार् ध	**गोल** अर्ध

❏ Constraining Dependent Vowels + BPE (CBPE): “Handling Indian script”

Algorithm 2 CBPE (Constrained BPE) Algorithm

Require: [Input] Training Corpus $\mathcal{C}$; Number of Merges $\mathcal{K}$
Ensure: [Output] Vocabulary $\mathcal{V}$, Merges $\mathcal{M}$

$\mathcal{V} \leftarrow \emptyset, \mathcal{M} \leftarrow \emptyset$
Initialize vocabulary with dependent vowels attached to preceding Unicode characters
while $|\mathcal{V}| < \mathcal{K}$ **do** ▷ Performing merges using the standard BPE algorithm
  $(t_l, t_r) \leftarrow$ Select the most frequent bigram pair in $\mathcal{C}$
  $\mathcal{V} \leftarrow \mathcal{V} \cup \{t_l t_r\}$
  $\mathcal{M} \leftarrow \mathcal{M} \cup \{(t_l, t_r)\}$
  Replace all occurrences of (t_l, t_r) with $t_l t_r$ in $\mathcal{C}$
end while

Word	BPE Initialization	CBPE Initialization
क़लम	क_◌़_ल_म	**क़**_ल_म
पढ़ाई	प_ढ_◌़_◌ा_ई	प_**ढ़ा**_ई
कार्यालय	क_◌ा_र_◌्_य_◌ा_ल_य	**का**_**र्**_**या**_ल_य

BharatGen's Tokenizer Fertility

Language	**AdaptMix** (Param)	Qwen	LLaMA	Nemotron Mistral	Nemotron Mini	Sarvam-M	DeepSeekv3	Gemma 27B	Airavata
Assamese	**1.93**	7.18	8.06	4.24	4.58	4.24	3.59	2.68	8.94
Bengali	1.90	6.92	7.85	2.93	2.65	2.93	2.89	**1.74**	8.20
English	1.47	1.36	1.35	1.37	1.35	1.37	1.33	**1.35**	1.58
Gujarati	**2.03**	8.53	9.54	3.59	15.17	3.59	4.84	2.39	14.14
Hindi	1.43	4.66	2.65	1.97	1.77	1.97	2.92	**1.38**	1.80
Kannada	**2.30**	11.08	13.81	3.82	4.02	3.82	5.83	3.15	19.35
Maithili	**1.73**	4.67	2.85	2.53	2.28	2.53	3.28	1.90	2.45
Malayalam	**2.60**	13.30	16.00	4.88	4.71	4.88	7.83	3.39	11.38
Marathi	**1.87**	6.46	3.86	3.14	2.62	3.14	4.15	1.94	3.31
Nepali	**1.70**	6.28	3.61	3.04	2.32	3.04	4.07	2.06	3.05
Oriya	1.95	12.92	15.91	**1.71**	17.24	17.23	7.26	4.60	17.29
Punjabi	**1.61**	7.39	7.88	3.12	12.70	3.12	4.51	2.74	10.84
Sanskrit	**2.57**	8.00	4.75	4.26	4.32	4.26	4.95	3.36	4.66
Sindhi	**1.67**	3.09	2.99	2.65	2.83	2.65	2.98	2.14	5.04
Tamil	**2.35**	9.75	11.89	3.71	3.57	3.71	4.88	2.42	10.50
Telugu	**2.34**	11.45	13.30	3.90	3.77	3.90	5.99	2.93	19.51
Average	**1.97**	**7.69**	**7.89**	**3.18**	**5.37**	**4.15**	**4.46**	**2.51**	**8.88**

Talk Overview

The Model Development Life Cycle

Case Study on BharatGen's Flagship Model: Param

- Training Dataset and Curation Process
- Tokeniser -> Algorithms -> Fertility Comparison
- Architecture Details

LLM Post-Training and Off-Policy, On-Policy Interplay

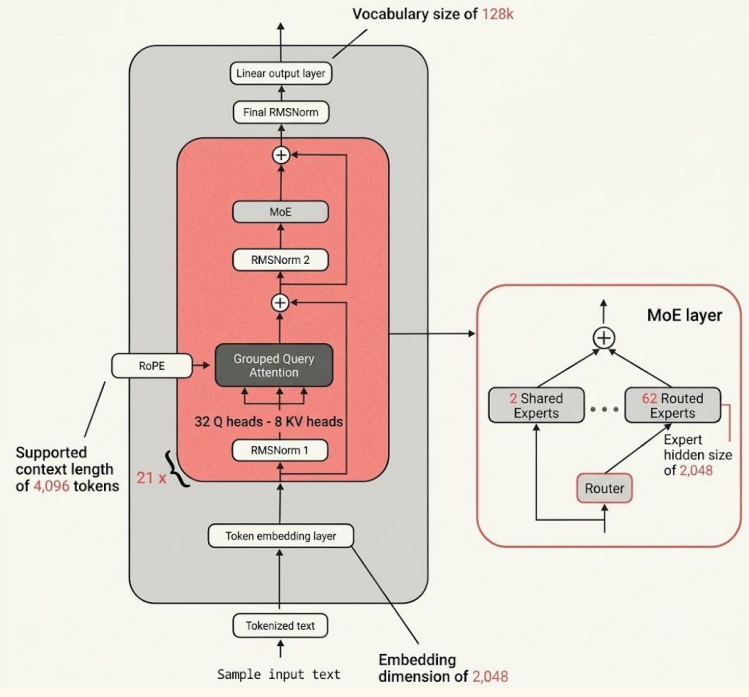
Provenance Mapping Key to Unlocking and Sustaining Performance Climb!

Graduating LLMs to Intelligent Agentic AI Systems: Harness/Agents

Our Current Flagship Model: Param-17B Mixture of Experts LLM

Param-2-17B-MOE-A2.4B: High-Efficiency Multilingual Powerhouse

State-of-the-art Mixture-of-Experts (MoE) model with 17B total parameters (2.4B active per token), combining massive knowledge capacity with fast, efficient inference, trained on ~22T tokens across 64 experts. Supports 22 Indian languages with strong code, math, and tool-use reasoning, optimized for culturally grounded, region-specific tasks and inclusive representation.



Languages supported		
• Assamese	• Kannada	• Punjabi
• Bengali	• Kashmiri	• Sanskrit
• Bodo	• Konkani	• Santali
• Dogri	• Maithili	• Sindhi
• Gujarati	• Malayalam	• Tamil
• Hindi*	• Manipuri	• Telugu
• Marathi	• Nepali	• Urdu
• Odia (Oriya)		

Now available for access through:

- **AIKosh:** Link
- **Hugging Face:** Link

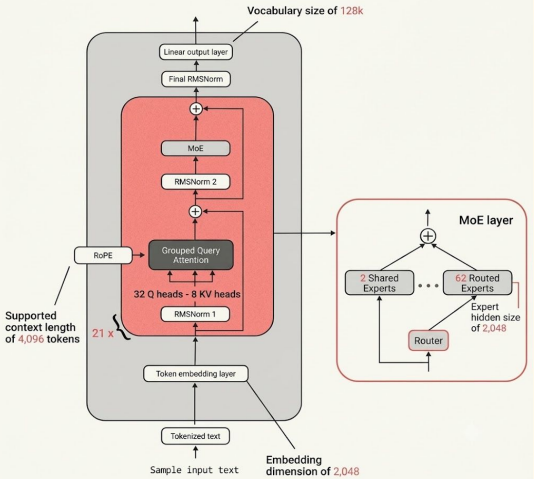
Our Current Flagship Model: Param-17B Mixture of Experts LLM

PARAM 2 17B MoE - Hybrid MoE: Efficiency & Specialization

Architectural Innovation

Employs a unique **Hybrid MoE** approach to solve the "instability" often found in sparse models:

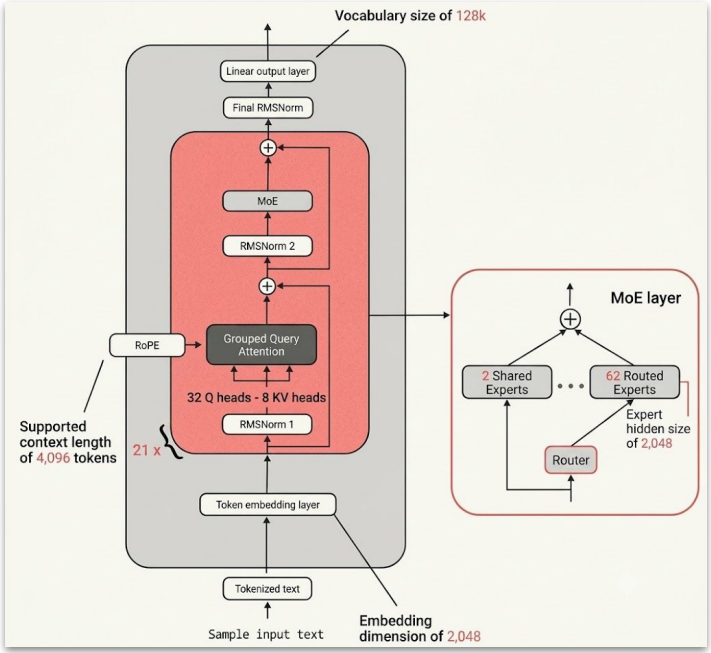
1. **Shared Experts:** Two experts remain **always active** to maintain stable cross-lingual representations and smooth code-switching.
2. **Routed Experts:** Six experts dynamically selected per token to handle specialized tasks and specific language nuances.
3. **Dense Pre-processing:** A dense layer transforms raw inputs into structured representations before MoE, ensuring better training stability.



Metric	Value	Metric	Value
Total Parameters	17B	Active Params/Token	2.4B
Total Experts	64	Top-K Experts	6 (Routed) + 2 (Shared)
Context Length	4096 Tokens	Vocabulary Size	128,000
Attention	Grouped Query (GQA)	Precision	bf16-mixed
Hidden Layers	21	Activation	SiLU / RMSNorm

Forward Looking: A long-context variant supporting up to **128k tokens** currently in development for long-form document processing.

Our Current Flagship Model: Param-17B Mixture of Experts LLM



Why Mixture-of-Experts?

Dense model

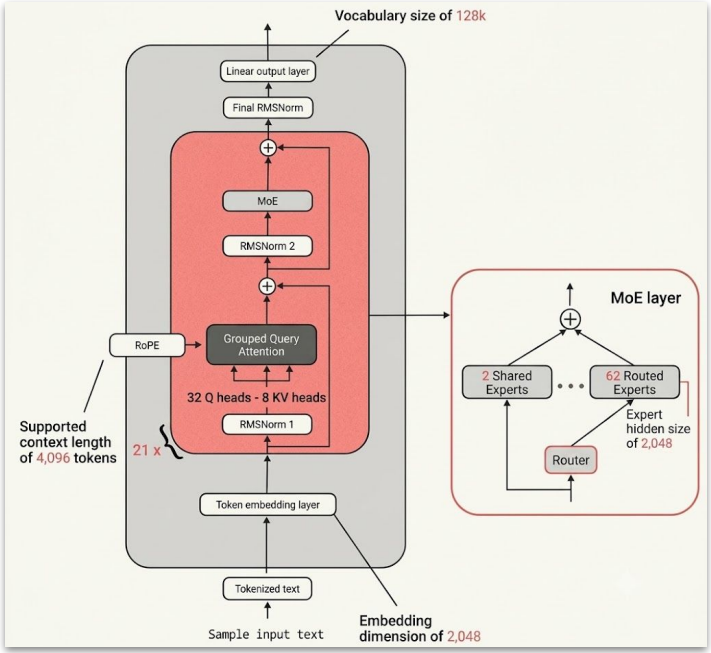
Every token passes through every parameter. Compute grows in step with model size.

Sparse MoE

Many expert FFNs exist, but a router sends each token to only a few. Capacity grows without matching compute.

- PARAM 2: 17B parameters stored, ~2.4B used per token (about 14%)
- Trade-off: more memory and routing complexity, less FLOPs per token
- Sparse models can be unstable; this design adds fixes (next slides)

Our Current Flagship Model: Param-17B Mixture of Experts LLM

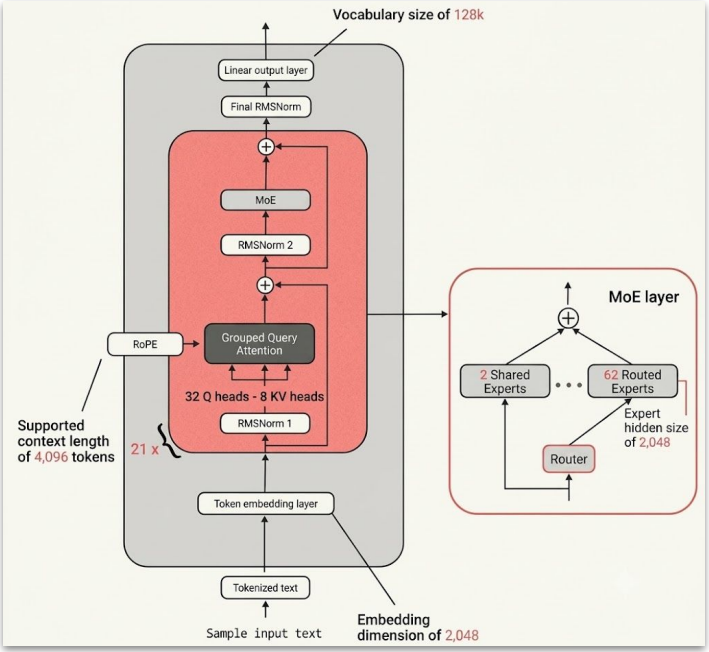


Model specifications

Metric	Value	Metric	Value
Total parameters	17B	Active params / token	2.4B
Total experts	64	Top-K experts	6 routed + 2 shared
Context length	4,096 tokens	Vocabulary size	128,000
Attention	Grouped Query (GQA)	Precision	bf16-mixed
Hidden layers	21	Activation	SiLU / RMSNorm

Also from the diagram: embedding dimension 2,048 · 32 Q heads / 8 KV heads · expert hidden size 2,048

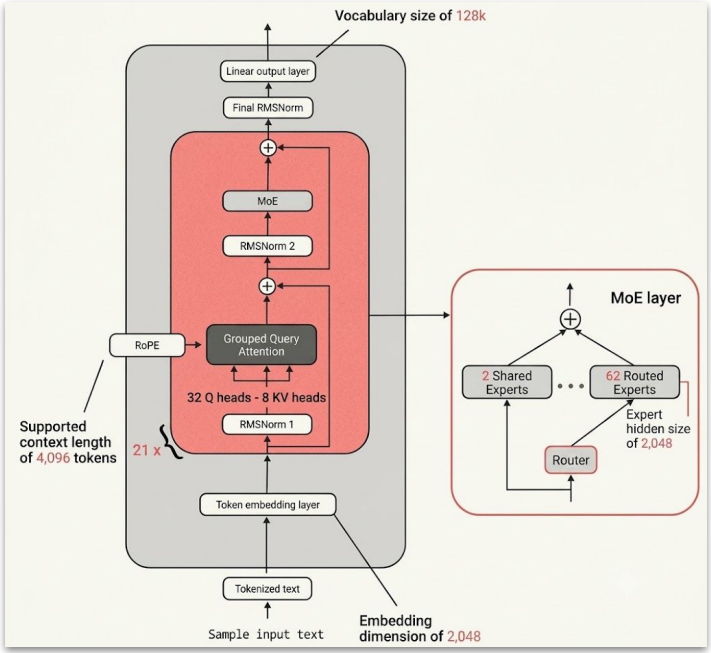
Our Current Flagship Model: Param-17B Mixture of Experts LLM



End-to-end data flow

1. **Tokenised text** → integer IDs (128k vocabulary)
2. **Token embedding layer** → vectors of dimension 2,048
3. **Layer 1: dense FFN** turns raw embeddings into structured representations
4. **Layers 2–21: sparse MoE**. Each layer is RMSNorm 1 → GQA (RoPE, QK norm) → residual add → RMSNorm 2 → MoE → residual add
5. **Final RMSNorm → Linear output layer** → LOGITS over 128k tokens

Our Current Flagship Model: Param-17B Mixture of Experts LLM



Dense first layer

What

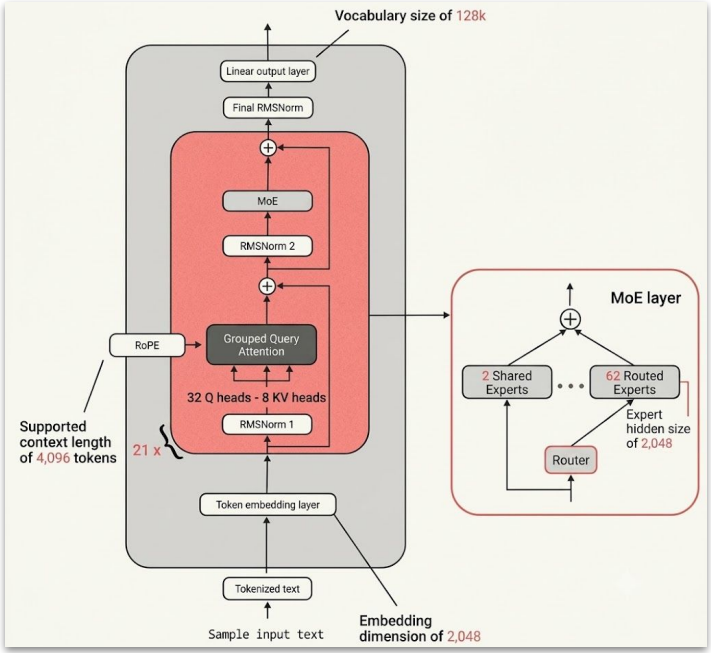
Layer 1 is a dense feed-forward layer. The other 20 layers use sparse MoE routing.

Why

It turns raw token embeddings into structured representations, a solid base before experts start to specialize.

- Improves stability in the early phase of training
- Routers get meaningful inputs rather than raw embeddings
- Together with shared experts, this is the model's stability toolkit

Our Current Flagship Model: Param-17B Mixture of Experts LLM



Grouped Query Attention (GQA)

MHA

32 Q, 32 K, 32 V heads. Largest KV cache.

MQA

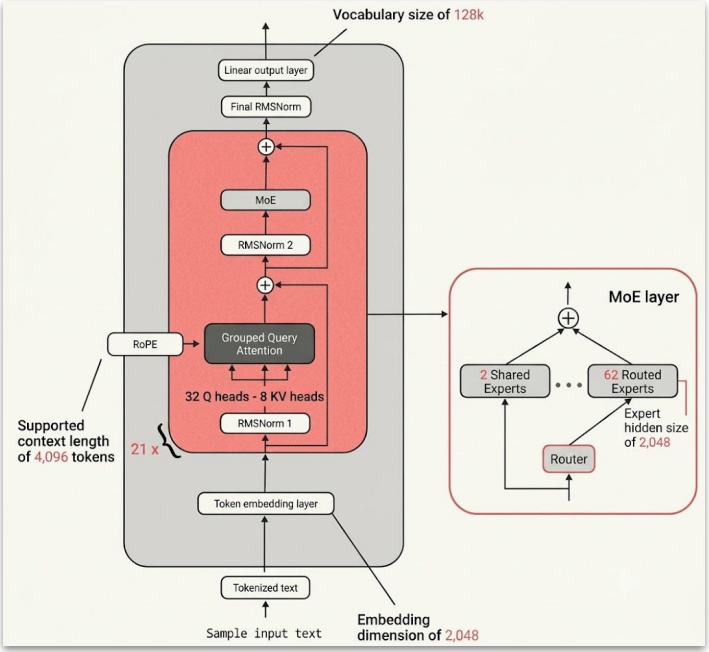
32 Q heads share 1 K/V head. Smallest cache, quality loss.

GQA (PARAM 2)

32 Q heads share 8 K/V heads: 4 Q heads per group. A middle ground.

- Head size = 2048 / 32 = 64
- KV cache is 4× smaller than full multi-head attention
- Faster long-context inference with little quality loss

Our Current Flagship Model: Param-17B Mixture of Experts LLM



The Hybrid MoE layer (layers 2–21)

2 Shared experts

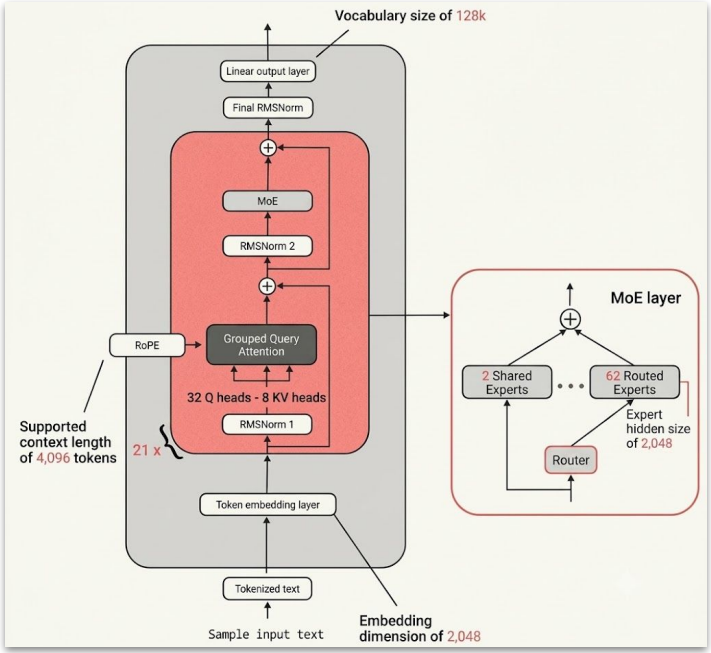
Always active, running in parallel with the routed experts. Hold common linguistic knowledge, which matters for a multilingual model.

62 Routed experts

The router picks the top 6 per token. Each specialises in particular tasks or language nuances.

Output = Σ shared experts + Σ (gate × routed expert) over the 6 chosen. 8 of 64 experts run per token; expert hidden size is 2,048.

Our Current Flagship Model: Param-17B Mixture of Experts LLM



Inside one transformer block

Pre-norm + residual

RMSNorm before attention and before MoE; each sub-layer output is added (⊕) back to its input, so gradients flow through 21 layers.

RMSNorm

x / RMS(x) · g. No mean subtraction, so it is cheaper than LayerNorm.

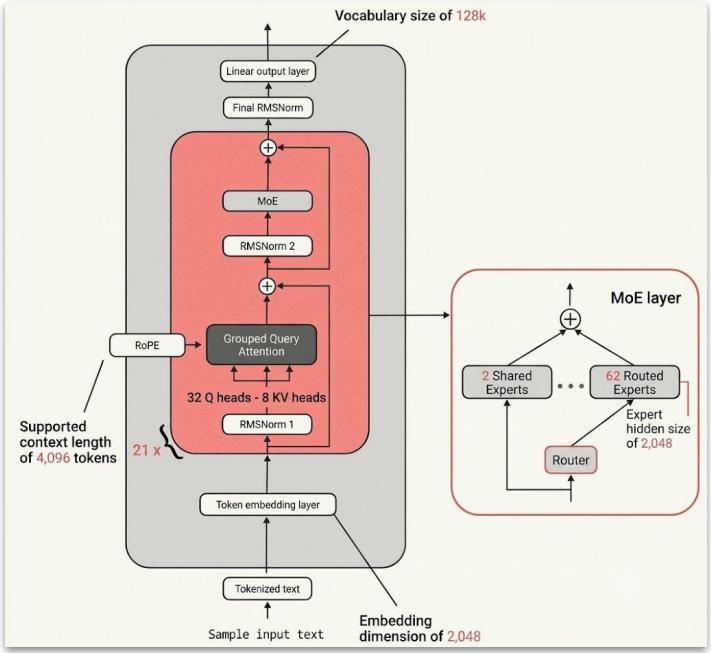
QK LayerNorm

Q and K are normalized (per head) before the dot product. This keeps attention logits bounded and prevents training spikes.

RoPE

Rotates Q and K by position-dependent angles, so attention depends on relative position. Supports 4,096 tokens.

Our Current Flagship Model: Param-17B Mixture of Experts LLM



Design lineage

Qwen-3 foundation

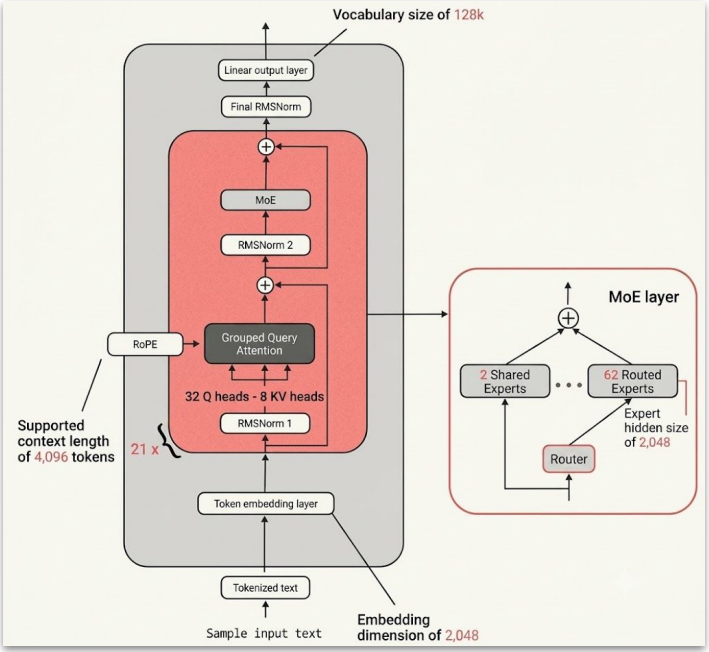
QK LayerNorm for training stability, RoPE, and a 128,000-token vocabulary covering all 22 Indian languages plus code and mathematical notation.

DeepSeek-inspired routing

Sigmoid routing with expert bias, always-on shared experts, and a sequence-level auxiliary loss.

- Param2 combines a proven dense backbone with proven sparse routing ideas
- Multilingual focus drives the vocabulary and the shared-expert design

Our Current Flagship Model: Param-17B Mixture of Experts LLM

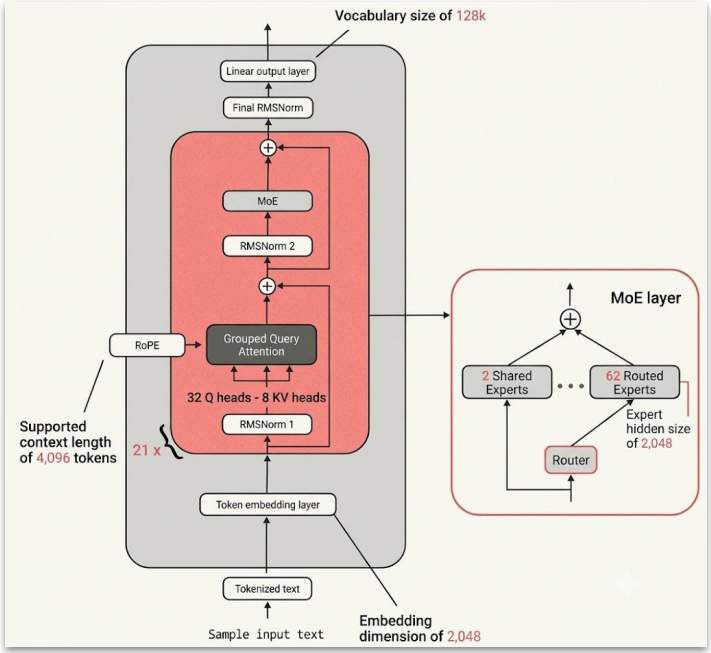


Sigmoid routing with expert bias

- Router: linear layer maps the 2,048-d token vector to 62 scores
- Scores use a **sigmoid** per expert, not softmax, so experts are not forced to compete for a fixed probability budget
- A **learnable bias per expert** shifts selection toward under-used experts
- Top-6 of the biased scores are chosen; shared experts bypass routing

Goal: prevent expert collapse, where a few experts receive most of the traffic.

Our Current Flagship Model: Param-17B Mixture of Experts LLM



Load balancing: sequence-level auxiliary loss

Token-level loss

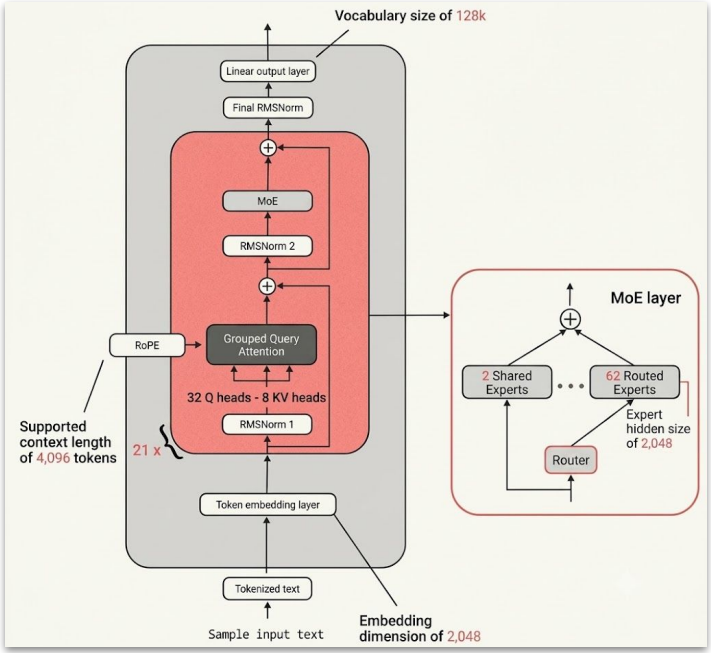
Balances expert usage over all tokens in a batch. Noisier signal and a sequence can still be lopsided.

Sequence-level loss

Balances expert usage within each sequence. Steadier gradient signals during training.

- Auxiliary loss is added to the language-modelling loss with a small weight
- Bias and auxiliary loss work together to avoid collapse

Our Current Flagship Model: Param-17B Mixture of Experts LLM



PARAM 2: routing and loss equations

One MoE layer (layers 2–21): 62 routed experts, top-6, plus 2 shared experts

A. FORWARD PASS

1. Router score (sigmoid)

$$s_{i,t} = \sigma\left(\mathbf{u}_t^\top \mathbf{e}_i\right), \qquad i = 1, \ldots, 62$$

2. Expert selection (bias used here)

$$\mathcal{T}_t = \mathrm{TopK}_6\left\{ s_{i,t} + b_i \right\}_{i=1}^{62}$$

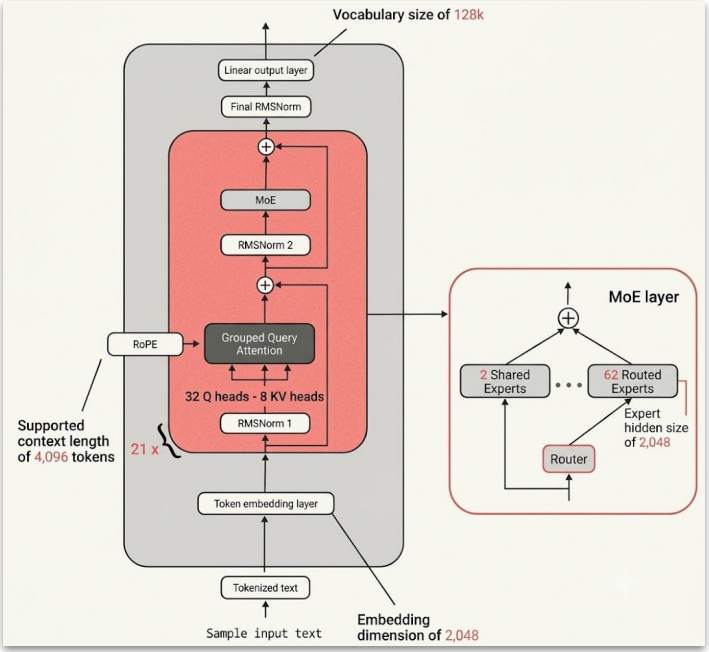
3. Gate value

$$g_{i,t} = \begin{cases} \dfrac{s_{i,t}}{\sum_{j \in \mathcal{T}_t} s_{j,t}}, & i \in \mathcal{T}_t \\ 0, & \text{otherwise} \end{cases}$$

4. Layer output (shared + routed)

$$\mathbf{h}_t = \mathbf{u}_t + \sum_{k=1}^{2} \mathrm{FFN}_k^{\mathrm{sh}}(\mathbf{u}_t) + \sum_{i=1}^{62} g_{i,t}\, \mathrm{FFN}_i(\mathbf{u}_t)$$

Our Current Flagship Model: Param-17B Mixture of Experts LLM



B. SEQUENCE-LEVEL LOAD BALANCING

5. Load fraction of expert i

$$f_i = \frac{62}{6\,T} \sum_{t=1}^{T} \mathbb{1}[i \in \mathcal{T}_t]$$

6. Mean routing probability of expert i

$$P_i = \frac{1}{T} \sum_{t=1}^{T} \frac{s_{i,t}}{\sum_{j=1}^{62} s_{j,t}}$$

7. Auxiliary balance loss (per sequence)

$$\mathcal{L}_{\text{bal}} = \sum_{i=1}^{62} f_i \, P_i$$

C. TRAINING OBJECTIVE

8. Language-modelling loss

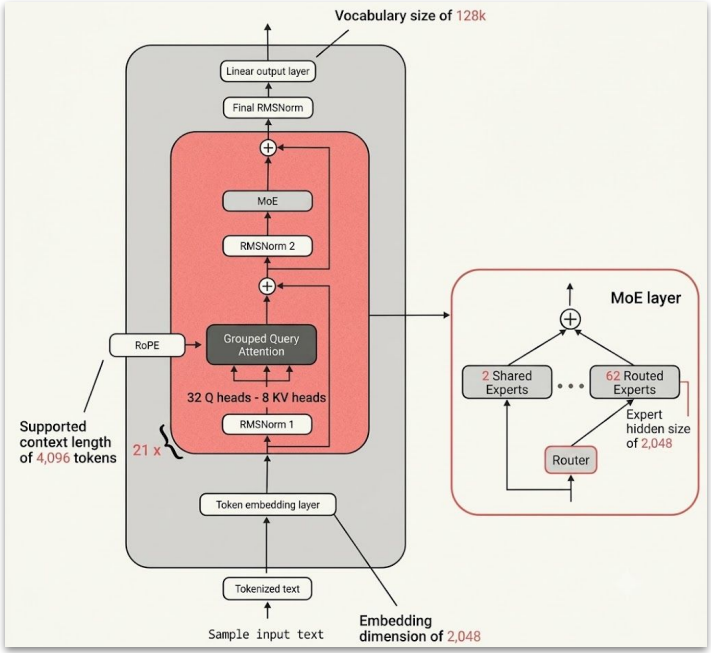
$$\mathcal{L}_{\text{LM}} = -\frac{1}{T} \sum_{t=1}^{T} \log p(x_t \mid x_{<t})$$

9. Total loss

$$\mathcal{L} = \mathcal{L}_{\text{LM}} + \alpha \, \mathcal{L}_{\text{bal}}$$

T: tokens in the sequence · $\mathbf{u}_t$: input to the MoE layer · $\mathbf{e}_i$: router vector of expert i · b_i: learnable expert bias · α: small weight. With perfect balance, $f_i = 1$ and $P_i = 1/62$ for all i. L_bal is averaged over sequences and the 20 MoE layers. Standard DeepSeek-style form; confirm Param2's exact normalization, whether b_i also enters the gate, and α.

Our Current Flagship Model: Param-17B Mixture of Experts LLM

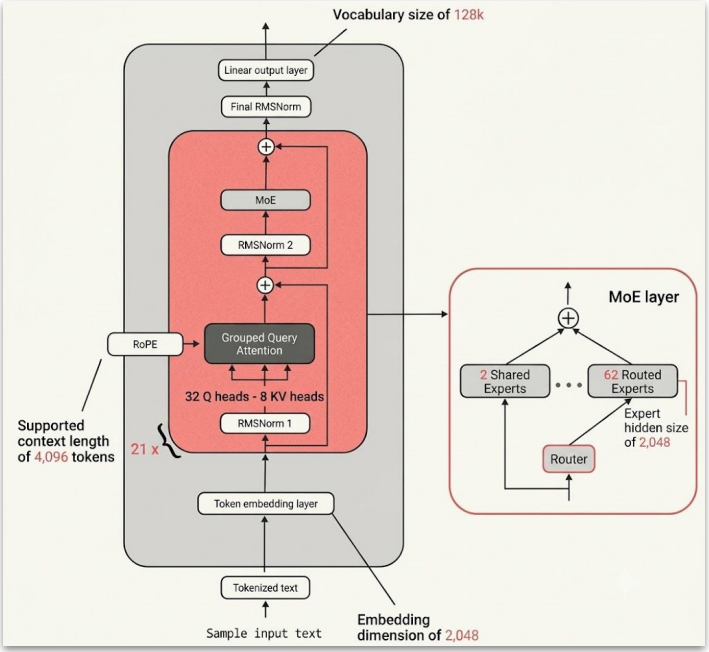


Dimension analysis: tensor shapes

B = batch, T = sequence length (up to 4,096), d = 2,048, head size = 64

Step	Shape
Token IDs	[B, T]
Embedding	[B, T, 2048]
Q = x·Wq (2048×2048)	[B, T, 2048] → [B, 32, T, 64]
K, V = x·Wk, x·Wv (2048×512)	[B, T, 512] → [B, 8, T, 64]
Scores per head	[B, 32, T, T]
Attention output ·Wo (2048×2048)	[B, T, 2048]
MoE: router 2048×62; each expert in 2048 → 2048 → 2048	[B, T, 2048]
Final linear (2048×128000)	[B, T, 128000]

Our Current Flagship Model: Param-17B Mixture of Experts LLM



Key terms

Term	Meaning
Total vs active params	Stored vs used per token
Top-K / expert / router	Experts chosen per token / a feed-forward network / the scorer
Shared vs routed expert	Always on vs selected per token
Expert collapse	A few experts get most tokens
Sigmoid routing + bias	Independent expert scores, bias for balance
Sequence-level aux loss	Balance penalty computed per sequence
GQA / RoPE / QK norm	Shared KV heads / rotary position / normalize Q and K
RMSNorm / SiLU	Root-mean-square norm / x · sigmoid(x)

So are we done with Training? ***Apparently, we've just begun!***

Pre-training: knowledge without behaviour

The only objective

$$L = -\Sigma_t \log p(x_t \mid x<t)$$

- Cross-entropy at every token: given a context, predict what comes next.
- Trained on trillions of tokens — the knowledge is unquestionably in there.
- But it lives spread across a probability distribution in hyper-dimensional space, not as callable behaviour.

✓ What you get

- World knowledge, language, code, facts
- Raw completion ability: continue any context plausibly

✗ What you don't get

- Instruction adherence — it completes, it does not obey
- Comprehension of a task, planning, building a blueprint
- All the "fancy stuff" you associate with ChatGPT / Claude

Talk Overview

The Model Development Life Cycle

Case Study on BharatGen's Flagship Model: Param

- Training Dataset and Curation Process
- Tokeniser -> Algorithms -> Fertility Comparison
- Architecture Details

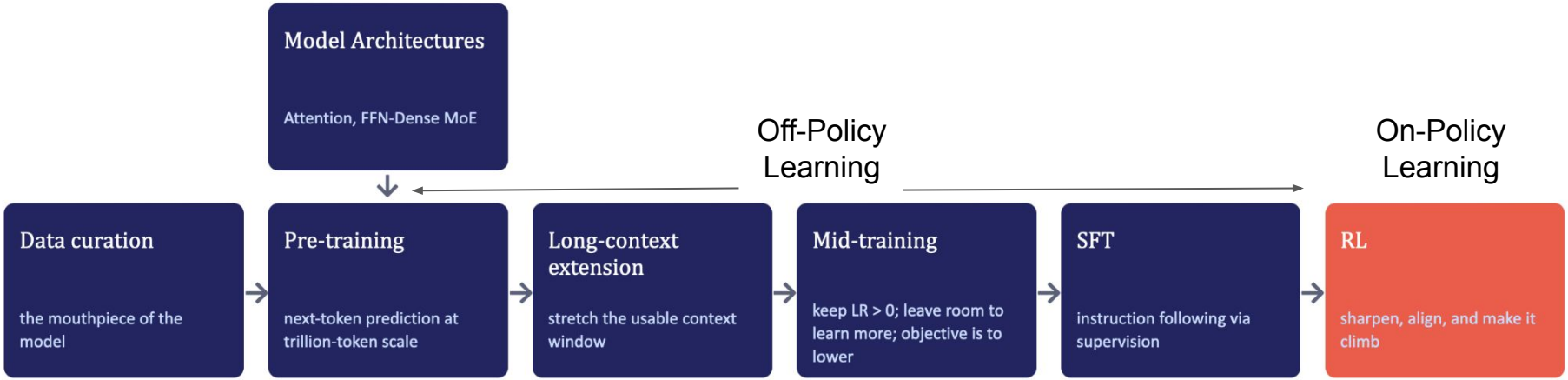
LLM Post-Training and Off-Policy, On-Policy Interplay

Provenance Mapping Key to Unlocking and Sustaining Performance Climb!

Graduating LLMs to Intelligent Agentic AI Systems: Harness/Agents

The Model Development Life Cycle: *Double Clicked!*

The LLM development lifecycle



Two things people underestimate

- Mid-training is a recent, load-bearing phase: a pre-training-style run between pre-training and post-training. Labs deliberately do not decay the learning rate to zero — that headroom is what makes mid-training, sometimes several rounds of it) possible.
- The pipeline is not one-way. If RL stalls, you go back — to SFT, or all the way to mid-training — and climb again.

SFT: same loss, two crucial changes

1 The data changes

(instruction, response) pairs in a chat template — the model learns the format of being asked and answering.

2 The loss is masked

Cross-entropy is computed on assistant tokens only. Prompt tokens are masked: learn to generate the whole answer given the context, not to predict the prompt.

One training example, token by token



grey = masked (no loss) **orange = loss computed**

Still token-level cross-entropy, summed over the response — the objective never changed, only what it is pointed at.

THE CATCH

After SFT, your model is still a coin toss

Ask it its own training question, 10 times

Sometimes it answers, sometimes it doesn't. The distribution is still loose — supervision showed it one path, it did not make that path reliable.

"Off-Policy" Distillation teaches confidence, not competence

Train on synthetic answers from DeepSeek or Claude, and the student imitates the teacher's tone of certainty — with or without the underlying ability.

Confidently "solves"

A fourth-order Schrödinger equation

...because that exact solution was in the SFT data.

Then fumbles

Pythagoras with sides 3 and 4

...because imitation does not generalise out of distribution.

Off-Policy vs On-Policy Learning

THE CORE DISTINCTION

Off-policy imitates. On-policy experiences.

OFF-POLICY · pre-training, SFT, off-policy distillation

Learn from someone else's trajectories

- You force another distribution onto the model; the obedient student imitates it.
- Great for breadth: new formats, new domains, new skills.
- Fragile out of distribution — the Pythagoras failure lives here.

ON-POLICY · reinforcement learning

Learn from your own rollouts

- The model generates, gets rewarded, and updates on its own behaviour.
- Don't hand it the solved equation — let it think harder from knowledge it already has from pre-training.
- Like solving problems at home without peeking at the back pages of the book.

Mnemonic: SFT expands what the model can express. RL sharpens what it can reliably do.

Talk Overview

The Model Development Life Cycle

Case Study on BharatGen's Flagship Model: Param

- Training Dataset and Curation Process
- Tokeniser -> Algorithms -> Fertility Comparison
- Architecture Details

LLM Post-Training and Off-Policy, On-Policy Interplay

Provenance Mapping Key to Unlocking and Sustaining Performance Climb!

Graduating LLMs to Intelligent Agentic AI Systems: Harness/Agents

Provenance Mapping key to unlocking and sustaining *"Performance Climb!"*

BEFORE RL CAN WORK

RL needs in-context learning — and ICL comes from scale

ICL is emergent

In-context learning builds up over training; empirically it strengthens with scale. Without it, on-policy exploration has nothing to steer with.

Two ways to scale

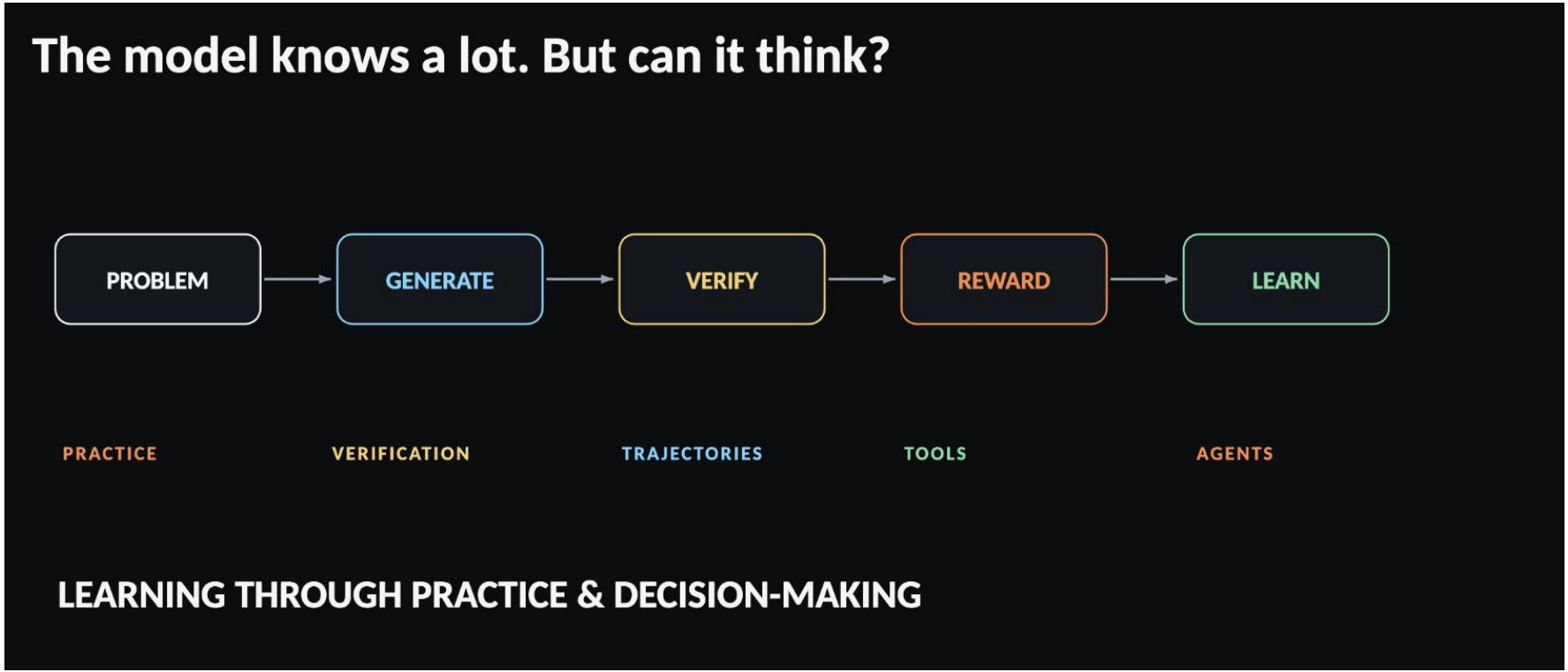
More parameters (a bigger brain) — or more FLOPs on the same brain. Both grow capability; only one grows your serving bill forever.

Overfitting? Please.

Success stories of SLMs over Trillions of Tokens of Training exists — performs good on everyday, and agentic tasks. If you manage to overfit an LLM, that itself is an achievement!

The efficiency thesis: you don't need an Einstein's brain if you learn efficiently. Parameter scaling helps — efficient learning substitutes for it.

Letting the Model Exercise its own Knowledge! – via RL



On-Policy Learning through Reinforcement Learning RL

One Model, Every Capability

Foundation-scale RL across a dozen expert domains — the capability surface we optimise Param for

Mathematical Reasoning

Competition-grade problem solving & proof writing

STEM & Scientific Reasoning

Graduate-level science, research QA & critical reasoning

Coding

From single functions to multi-file repositories

Agentic & Software Engg.

Planning, executing and fixing real pull requests

Search & Tool Use

Multi-hop research with live tools & browsing

Terminal & Environment Use

Operating shells, sandboxes and real workflows

Instruction Following & Writing

Precise, faithful, creative generation

Multilingual Intelligence

22 Indian languages + English, code-switched

Safety & Abstention

Knowing what to answer — and what not to

UP UNTIL NOW, **Imitation to Reasoning to Agentic Capabilities**

NEXT, **Intelligent Agentic AI Systems**

Talk Overview

The Model Development Life Cycle

Case Study on BharatGen's Flagship Model: Param

- Training Dataset and Curation Process
- Tokeniser -> Algorithms -> Fertility Comparison
- Architecture Details

LLM Post-Training and Off-Policy, On-Policy Interplay

Provenance Mapping Key to Unlocking and Sustaining Performance Climb!

Graduating LLMs to Intelligent Agentic AI Systems: Harness/Agents

Graduating LLMs to Intelligent Agent AI Systems

Intelligence Alone Doesn't Act

The Harness is what connects a model's intelligence to a real environment and a real task

LLM

The Intelligence

- Knowledge & reasoning
- Language understanding
- Raw problem-solving ability
- No hands, no eyes, no tools of its own

HARNESS

The Nervous System

- Agent loop (plan → act → observe)
- Memory across steps & sessions
- Tool signatures & interfaces
- Turns intelligence into action

ENVIRONMENT

Where the Task Lives

- GitHub repos / Docker sandboxes
- Legal & compliance systems
- Financial workflows
- Healthcare & enterprise systems

The environment changes with the user. The harness is what lets the same model act inside any of them.

Thank You